

IFRI-UAC

Rapport de projet

TP1 Big Data

Recommendation system: Collaborative Filtering item-item top-N

Informations etudiant

Nom et prenom :	AGBESSI John Ulrich
Filiere :	Genie logiciel
Ecole :	IFRI-UAC
Depot GitHub :	github.com/John271200/TP1Recommendation-system
Application :	tp1recommendation-system.streamlit.app

Projet realise dans le cadre du TP1 : Recommendation system avec Streamlit

Resume

Ce rapport presente la conception et la realisation d'un systeme de recommandation top-N base sur le filtrage collaboratif item-item. Le systeme exploite les notes donnees par plusieurs utilisateurs a des films, calcule les similarites entre les items, puis recommande les films les plus pertinents pour un utilisateur cible.

Liens du projet

Depot GitHub	https://github.com/John271200/TP1Recommendation-system.git
Application	https://tp1recommendation-system.streamlit.app/
Acces prof.	johnaoga@gmail.com

1. Introduction

Les systemes de recommandation sont presents dans de nombreux services numeriques : plateformes de streaming, commerce electronique, reseaux sociaux et bibliotheques numeriques. Leur role est d'aider l'utilisateur a trouver rapidement des contenus pertinents.

Dans ce TP, l'approche utilisee est le filtrage collaboratif item-item. Elle compare les items entre eux a partir des notes donnees par les utilisateurs. Deux films sont proches si les utilisateurs les evaluent de maniere similaire.

2. Objectifs du projet

- Lire et exploiter un dataset utilisateur-item.
- Construire les structures de donnees necessaires au filtrage collaboratif.
- Calculer la similarite cosinus entre les items.
- Predire les scores des items non notes par un utilisateur.
- Afficher une liste top-N de recommandations.
- Proposer une interface web interactive avec Streamlit.
- Publier le projet sur GitHub et le deployer en ligne.

3. Donnees utilisees

Le projet utilise deux fichiers CSV. Le fichier ratings.csv contient les notes donnees par les utilisateurs. Le fichier items.csv associe les identifiants des items aux titres des films.

Fichier	Role	Colonnes
ratings.csv	Notes utilisateur-item	user_id, item_id, rating
items.csv	Description des films	item_id, title

Le dataset contient 8 utilisateurs, 8 films et 32 notes. Il est volontairement reduit pour faciliter l'analyse du fonctionnement de l'algorithme.

4. Architecture du projet

```
TP1/  
  app.py  
  recommender.py  
  ratings.csv  
  items.csv  
  requirements.txt  
  README.md  
  EXECUTION_SUMMARY.md  
  SUBMISSION_LINKS.md  
  RAPPORT_TP1_AGBESSI_John_Ulrich.pdf
```

Fichier	Description
recommender.py	Logique de recommandation item-item
app.py	Interface web Streamlit
ratings.csv	Notes des utilisateurs
items.csv	Titres des films
requirements.txt	Dependance Streamlit

5. Methode de recommandation

5.1 Filtrage collaboratif item-item

Le filtrage collaboratif item-item cherche les items similaires aux items deja apprecies par l'utilisateur. Cette approche est interpretable : un film est recommande parce qu'il est proche d'autres films deja notes.

5.2 Structures de donnees

- ratings_by_user : dictionnaire permettant de retrouver les notes d'un utilisateur.
- ratings_by_item : dictionnaire permettant de retrouver les notes recues par un item.

5.3 Similarite cosinus

Chaque item est represente par un vecteur de notes. Les notes manquantes sont remplacees par 0.

$$\text{similarite}(A, B) = (A \cdot B) / (||A|| * ||B||)$$

5.4 Prediction des scores

Pour chaque item candidat, le score predit est calcule avec une moyenne ponderee par les similarites.

```
score(item) = somme(similarite(item, item_note) * note) / somme(abs(similarite))
```

6. Interface Streamlit

L'interface Streamlit rend le systeme accessible depuis un navigateur. Elle permet de choisir l'utilisateur cible, de regler le nombre de recommandations, de consulter les notes deja donnees et de visualiser les similarites item-item.

- Selection de l'utilisateur cible.
- Choix du nombre de recommandations top-N.
- Affichage des notes de l'utilisateur.
- Affichage des recommandations.
- Affichage des meilleures similarites item-item.

7. Execution du projet

7.1 Installation

```
python -m pip install -r requirements.txt
```

7.2 Lancement Streamlit

```
streamlit run app.py
```

7.3 Execution console

```
python recommender.py --user U1 --top-n 3 --show-similarities
```

8. Resultats obtenus

```
Top similarites item-item
John Wick <-> Mad Max Fury Road : 0.986
The Matrix <-> Mad Max Fury Road : 0.977
The Matrix <-> John Wick : 0.962
Inception <-> The Martian : 0.898
Interstellar <-> The Martian : 0.828
```

```
Recommandations top-3 pour U1
1. Gravity (I7) - score predit : 4.27
2. Blade Runner 2049 (I8) - score predit : 2.69
3. John Wick (I4) - score predit : 1.94
```

Pour l'utilisateur U1, les films déjà notés sont exclus des recommandations. Gravity obtient le meilleur score prédit, ce qui indique que ce film est le plus pertinent selon les similarités calculées avec les films déjà notés par l'utilisateur.

9. Déploiement

Le dépôt GitHub et l'application Streamlit sont disponibles en ligne.

Informations de soumission

GitHub	https://github.com/John271200/TP1Recommendation-system.git
Streamlit	https://tp1recommendation-system.streamlit.app/
Professeur	johnaoga@gmail.com

10. Limites et perspectives

10.1 Limites

- Le dataset est petit et sert principalement à illustrer l'algorithme.
- Les notes manquantes sont remplacées par 0 pour simplifier le calcul.
- Le problème de cold start n'est pas encore traité.
- Le modèle n'est pas encore évalué avec des métriques comme RMSE, précision ou recall.

10.2 Améliorations possibles

- Utiliser un dataset plus grand comme MovieLens.
- Ajouter une séparation train/test.
- Comparer les approches item-item et user-user.
- Ajouter l'import de fichiers CSV depuis Streamlit.
- Améliorer l'interface avec des graphiques et une matrice de similarité.

11. Conclusion

Ce TP a permis de réaliser un système de recommandation top-N complet, depuis la lecture des données jusqu'au déploiement d'une interface web. Le filtrage collaboratif item-item offre une approche claire et interprétable pour comprendre les bases des systèmes de recommandation.

L'application Streamlit complète le projet en rendant les recommandations facilement testables. Le dépôt GitHub et le lien de déploiement permettent au professeur de consulter le code et d'exécuter l'application en ligne.